

PKU Sleeper 睡眠打卡助手项目报告

一、程序功能介绍

PKU Sleeper 是一款面向北京大学学生的睡眠打卡助手，目标是帮助用户记录睡眠、分析作息、制定睡眠目标，并通过成就和校园地图解锁机制提高持续使用的积极性。程序基于 PySide6 构建图形界面，使用本地 JSON 文件保存用户数据，并结合 pandas、pyqtgraph 完成课表解析和数据可视化。

程序主要功能如下：

1. 睡眠打卡：用户可在首页开始或结束一次睡眠记录，并选择夜间睡眠或午休、宿舍/家/其他等睡眠环境。系统保存开始时间、结束时间、睡眠类型、目标时长等信息。
2. 睡眠记录管理：记录页展示近 7 天或近 30 天睡眠历史，包括日期、睡眠时长、起止时间和质量评分。
3. 睡眠报告分析：报告页统计平均睡眠时长、平均入睡时间、平均起床时间、目标完成率和平均评分，并绘制睡眠趋势图，生成阶段性文字总结。
4. 目标设置：目标页支持设置每日目标睡眠时长、预期入睡时间、预期起床时间和午休目标，并展示本周完成进度。
5. 成就系统：系统内置多项睡眠成就，如首次记录、8 小时睡眠、连续达标、午休养成等，并根据历史记录自动判断解锁情况。
6. 校园睡眠地图：地图页以北大校园地标为节点，根据累计记录、睡眠总时长、连续达标天数等条件逐步解锁红楼、西校门、博雅塔、图书馆等节点。
7. 作息规划：用户可上传或手动编辑课表，系统结合课程安排和睡眠目标，推荐夜间睡眠时间、午休日期和常用地点。
8. 用户信息与界面导航：主窗口提供侧边栏页面切换、个人设置入口和日期显示，形成完整的桌面应用体验。

二、项目模块与类设计细节

项目采用分层设计，将数据模型、业务逻辑、状态管理、持久化和界面展示相互分离。整体流程为：`main.py` 创建应用、仓库和 `MainTracker`，主窗口通过 `ServiceBridge` 调用业务服务，各页面控制器只负责界面刷新和事件绑定。

程序结构图如下：

```

main.py
└─ MainWindowController
    │   └─ UI 页面控制器
    │       └─ HomeController
    │       └─ RecordsController
    │       └─ ReportController
    │       └─ PlanningController
    │       └─ MapController
    │       └─ GoalController
    │       └─ AchievementController
    └─ ServiceBridge
        │   └─ MainTracker
        │       └─ SleepManager
        │       └─ GoalManager
        │       └─ AchievementManager
        │       └─ SleepMapManager
        │       └─ UserManager
        │   └─ domain/: 睡眠记录、目标、成就、地图节点等模型
        │   └─ states/: 睡眠状态、成就状态、地图状态
        │   └─ storage/: 本地 JSON 数据读写
        │   └─ reports/: 睡眠评分与报告生成
        └─ achievements/: 默认成就目录

```

1. 入口与总体控制

`main.py` 是程序入口，负责创建 `QApplication`，初始化 `SleepRecordRepository`、`SleepReportBuilder` 和 `MainTracker`，加载默认成就目录，并启动 `MainWindowController`。

`MainTracker` 位于 `pkusleeper/services/tracker.py`，是业务层的总门面。它组合了 `SleepManager`、`GoalManager`、`AchievementManager`、`SleepMapManager` 和 `UserManager`，统一对外提供开始睡眠、结束睡眠、生成报告、获取快照等接口。用户结束睡眠后，`MainTracker.wake_up()` 会依次完成记录保存、成就判断、目标判断和地图节点判断。

2. 模型层

模型层位于 `pkusleeper/domain/`，主要使用 `dataclass` 表示核心数据。

- `SleepInterruption`：表示一次睡眠中断，包含中断开始时间、结束时间和原因。
- `SleepSessionDraft`：表示正在进行的睡眠草稿，结束前可变，保存用户、开始时间、目标时长、睡眠类型、睡眠环境和中断列表。
- `SleepRecord`：表示最终写入历史记录的数据，包含唯一记录编号、起止时间、目标快照、睡眠类型、环境和中断信息。

- `SleepReport` : 表示报告结果, 包含原始记录、实际睡眠时长、中断次数、质量评分和文本摘要。
- `SleepGoal` : 表示睡眠目标, 包括目标时长、预期入睡时间、难度等级和午休目标。
- `SleepAchievement` : 表示成就规则, 通过 `demands` 字典描述解锁条件, 并用 `fulfilled_by()` 判断单条记录是否满足要求。
- `Node` : 表示校园地图节点, 保存节点编号、名称、描述和解锁条件。
- `User` 、 `Roommate` : 保存用户和室友信息。
- `SleepType` 、 `SleepEnvironment` : 用枚举限制睡眠类型和睡眠环境, 减少字符串错误。

这一层只描述数据结构和少量规则, 不直接处理界面或文件读写。

3. 服务层

服务层位于 `pkusleeper/services/` , 负责组织业务流程。

- `SleepManager` : 管理一次睡眠的完整生命周期。 `start_sleeping()` 创建睡眠草稿和 `SleepingState` , `interrupt_sleep()` 与 `continue_sleeping()` 记录中断, `wake_up()` 结束睡眠并生成 `SleepRecord` , 必要时调用仓库保存。
- `GoalManager` : 读取、保存和评估当前睡眠目标, 判断本次睡眠是否达到目标时长。
- `AchievementManager` : 维护成就列表和已解锁成就, 调用状态对象判断新记录是否触发成就。
- `SleepMapManager` : 管理地图节点列表和已解锁节点, 判断最新记录是否带来新的地图进度。
- `UserManager` : 维护用户昵称、室友列表、经验等级和课表数据。

服务层的设计使核心规则可以独立于 UI 使用, 后续也便于扩展命令行工具或其他前端。

4. 状态管理层

状态对象位于 `pkusleeper/states/` 。

- `State` 是抽象基类, 约定所有状态对象都提供稳定的 `name()` 。
- `SleepingState` 管理一场正在进行的睡眠, 包括当前中断、恢复睡眠和最终转换为 `SleepRecord` 。
- `AchievementState` 保存全部成就和已解锁成就, 并提供用户成就列表和新成就判断。
- `MappingState` 保存校园地图节点状态, 提供地图列表、解锁判断和节点详情查询。

状态层把“当前处于什么阶段”与业务服务分开, 使睡眠流程更清晰。

5. 持久化与报告生成

`SleepRecordRepository` 位于 `pkusleeper/storage/repository.py`，负责本地 JSON 数据读写。它按用户创建数据目录，支持保存、读取、删除和清空睡眠记录，也支持保存当前睡眠目标和开发调试状态。日期时间、枚举和中断列表在写入前会转换为可序列化格式，读取时再恢复为领域对象。

`SleepReportBuilder` 位于 `pkusleeper/reports/builder.py`，负责计算单条睡眠记录的质量评分并生成报告对象。夜间睡眠评分主要考虑睡眠时长、入睡时间和中断情况；午休评分主要考虑午休时长是否合理。报告页进一步基于多条记录统计平均时长、趋势变化和目标完成率。

6. 成就目录与地图规则

`pkusleeper/achievements/catalog.py` 中定义默认成就目录，成就条件包含单次时长、睡眠环境、中断次数、累计记录数、连续达标天数、平均睡眠时长等。

校园地图规则集中定义在 `ServiceBridge.MAP_NODES` 和 `MapBridgeMixin.MAP_REQUIREMENTS` 中。每个节点对应一个北大地标，并设置累计记录数、夜间睡眠次数、午休次数、累计睡眠时长、连续达标天数等解锁条件。`MapBridgeMixin` 会根据历史记录计算当前统计值，生成节点状态、解锁条件和进度文本。

7. UI 层与桥接层

UI 代码位于 `pkusleeper/ui/`。项目使用 Qt Designer 的 `.ui` 文件搭建页面结构，再由 Python 控制器绑定事件和刷新数据。

- `MainWindowController`：主窗口控制器，加载侧边栏和各子页面，维护 `QStackedWidget` 页面切换，并定时刷新公共日期栏。
- `UiController`：页面控制器基类，封装查找控件、设置文本、连接按钮、显示消息等通用方法。
- `ServiceBridge`：UI 层使用的薄门面，由多个 `mixin` 组合而成。它把业务对象转换成界面需要的字典数据，降低页面控制器与业务层的耦合。
- `HomeController`：控制首页，处理开始/结束睡眠、快捷入口和状态栏刷新。
- `RecordsController`：动态渲染历史睡眠记录列表。
- `ReportController` 与 `ReportChartMixin`：展示统计卡片和图表，支持 7 天/30 天切换。
- `GoalController`：处理睡眠目标设置、保存和本周进度展示。
- `AchievementController`：展示已解锁/未解锁成就、积分、等级和进度条。
- `MapController`：加载北大地图图片，按比例放置地标标记，并显示节点解锁状态。
- `PlanningController`：处理课表上传、手动编辑和作息规划结果展示。
- `ProfileController`：处理个人设置页面。

桥接层分为 `HomeSleepBridgeMixin`、`RecordsBridgeMixin`、`ReportsBridgeMixin`、`GoalsBridgeMixin`、`AchievementsBridgeMixin`、`MapBridgeMixin`、`PlanningBridgeMixin` 和 `BridgeCommonMixin`。这种 `mixin` 组合方式使不同页面的数据逻辑相对独立，同时统一由 `ServiceBridge` 暴露给 UI。

三、小组成员分工情况

成员	主要分工
胡雨轩	负责项目总体架构与功能设计、UI设计与制作、service实现等
罗午阳	负责UI控制系统及桥接编写、state实现等
肖 涵	负责数据分析、模型构建、storage实现等
共 同	素材整理、测试调试、README 与项目报告整理等

项目开发过程中，小组成员共同参与需求讨论、界面联调和最终测试。各模块之间通过明确接口协作，减少了多人开发时的相互影响。

四、项目总结与反思

本项目完成了一个具有完整界面和核心业务流程的睡眠打卡助手。项目的主要收获包括：

- 1. 分层设计提高了代码可维护性。领域模型、服务层、状态层、持久化层和 UI 层职责较清楚，便于分别开发和调试。
- 2. 数据驱动的规则设计具有扩展性。成就条件和地图节点条件主要通过字典描述，后续增加新成就或新地标时不需要大幅修改流程代码。
- 3. 图形界面增强了项目完整度。PySide6、Qt Designer、pyqtgraph 和图片素材共同构成了较直观的桌面应用体验。
- 4. 本地 JSON 存储实现简单可靠，适合课程项目规模，也方便检查和调试历史数据。

项目也存在一些可以继续改进的地方：

- 1. 目前用户体系较简单，只支持默认用户，本地数据目录也未提供完整的多用户切换界面。
- 2. 睡眠质量评分规则仍偏经验化，可以进一步结合更多指标或用户反馈进行调整。
- 3. 成就和地图的部分聚合规则目前主要在桥接层统计，后续可下沉到服务层，使业务逻辑更加集中。
- 4. 当前测试以开发命令和人工联调为主，自动化单元测试覆盖还可以进一步补充。

5. 课表解析已兼容常见 Excel 格式，但面对非标准格式时仍可能失败，后续可以增加更多容错提示。

总体而言，PKU Sleeper 将课程中学习到的面向对象设计、文件读写、图形界面开发和模块化工程组织结合起来，形成了一个功能完整、主题明确、可继续扩展的课程大作业项目。